GATEWay

An Intelligent GATE DA Preparation Platform

With Advanced Recommendation Engines, Analytics, and Predictive Scoring

Khethavath Sunil Naik
12341170

April 14, 2026

Abstract

GATEWay is a sophisticated, classroom-aware educational intelligence platform designed specifically for GATE (Graduate Aptitude Test in Engineering) Data Analytics (DA) preparation. The system provides dual-mode experiences: a comprehensive student workspace for structured practice, adaptive testing, personalized coaching, and performance tracking, coupled with an advanced teacher command center for classroom orchestration, assignment management, learner monitoring, and intervention planning.

The platform's core innovation lies in its unified architecture that seamlessly integrates multiple specialized recommendation engines, a telemetry-driven analytics layer powered by Supabase, graph-based adaptive sequencing, intelligent rule-based coaching, comprehensive performance analytics, and a novel predictive scoring model. This report provides a detailed technical exposition of the complete system architecture, mathematical formulas underlying key recommendations, all data models, recommendation engine implementations, teacher analytics pipelines, and the comprehensive GATE score prediction formula with detailed variable explanations. The document spans multiple dimensions: product vision, system design, algorithmic foundations, implementation details, and research assets, making it suitable for technical portfolios, advanced project vivas, research presentations, and academic evaluation.

Contents

1 Introduction and Problem Statement **5**

1.1 Educational Challenges in GATE DA Preparation . . . 5

1.2 Platform Vision . . . 5

2 System Architecture and Design Philosophy **5**

2.1 Architecture Overview . . . 6

2.2 Technology Stack . . . 6

2.3 Multi-Schema Data Modeling Philosophy . . . 6

3 Data Model and Core Entities **7**

3.1 Public Schema Tables . . . 7

3.1.1 profiles . . . 7

3.1.2 user_progress . . . 7

3.1.3 answered_questions . . . 7

3.1.4 test_history . . . 8

3.1.5 activity_events . . . 8

3.2 Teacher Schema Tables . . . 8

3.2.1 teachers . . . 8

3.2.2 courses . . . 8

3.2.3 enrollments . . . 9

3.2.4 assignments . . . 9

3.2.5 submissions . . . 9

4 Student Learning Flow and Experience **9**

4.1 Onboarding and Authentication . . . 9

4.2 Enrollment and Course Joining . . . 9

4.3 Learning Modes . . . 9

4.3.1 Full Mock Test Mode . . . 10

4.3.2 Topic-Wise Practice Mode . . . 10

4.3.3 Adaptive Practice Mode . . . 10

4.3.4 Assignment Mode . . . 10

4.4 Performance Tracking and Metrics . . . 11

5 Adaptive Graph-Based Recommendation Engine **11**

5.1 Design Objectives and Philosophy . . . 11

5.2 Question Graph Construction . . . 11

5.2.1 Edge Types and Semantics . . . 11

5.3 Learner State Representation . . . 12

5.4 Target Policy Computation . . . 12
5.4.1 Target ELO . . . 12
5.4.2 Target Difficulty . . . 12
5.5 Candidate Selection and Neighborhood Navigation . . . 13
5.5.1 Remediation Neighborhood . . . 13
5.5.2 Current Question Neighborhood . . . 13
5.6 Candidate Scoring Function . . . 13
5.7 Output and Explainability . . . 14

6 Remediation and Retry Engine **14**
6.1 Remediation Workflow . . . 14
6.2 Remediation Exit Criteria . . . 15

7 Practice Review and Rapid-Guess Detection Engine **15**
7.1 Motivation: Quality of Evidence . . . 15
7.2 Threshold Computation . . . 15
7.3 Penalty Application . . . 15

8 Coach Engine **15**
8.1 Purpose and Scope . . . 15
8.2 Rule-Based Implementation Architecture . . . 16
8.3 Coach Input State . . . 16
8.4 Deterministic Coach Logic . . . 16
8.4.1 Daily Study Suggestion . . . 17
8.4.2 7-Day Study Plan . . . 17
8.5 Coach Reasoning Logic . . . 17
8.5.1 Step 1: Student Profile Analysis . . . 17
8.5.2 Step 2: Weakness Severity Ranking . . . 17
8.5.3 Step 3: Personalized Recommendations . . . 17
8.5.4 Step 4: Study Plan Generation . . . 18

9 Teacher Analytics and Intervention Intelligence **18**
9.1 Teacher Workspace Aggregation . . . 18
9.2 Teacher Summary Computations . . . 18
9.2.1 Student Summaries . . . 18
9.2.2 Course Summaries . . . 18
9.3 Risk Scoring . . . 19
9.4 Intervention Recommendations . . . 19

10 GATE Score Prediction Engine **19**
10.1 Vision and Purpose . . . 19

10.2 Unlock Criteria . . . 20
10.3 Core Formula: Predicted Score Basis . . . 20
10.3.1 Component 1: Overall Accuracy (Acc) . . . 20
10.3.2 Component 2: ELO Rating (ELO_{norm}) . . . 21
10.3.3 Component 3: Subject Consistency ($Cons_{norm}$) . . . 21
10.3.4 Component 4: Improvement Factor ($Impr_{norm}$) . . . 22
10.4 Score Band Generation from Basis . . . 22
10.4.1 Expected Score . . . 22
10.4.2 Minimum and Maximum Score Bands . . . 23
10.4.3 Confidence Score . . . 23
10.4.4 Recommendation Text . . . 23
10.5 Worked Example . . . 23

11 Telemetry, Persistence, and Event-Driven Observability 24
11.1 State Persistence Strategy . . . 24
11.1.1 Persistent State (Tables) . . . 24
11.1.2 Event Stream (activity_events table) . . . 24
11.2 Why This Design Matters . . . 25

12 History Review and Explainable Replaying 25

13 Research Assets and Analytical Notebooks 26

14 Testing and Quality Assurance 26
14.1 Test Coverage for Adaptive Engine . . . 26
14.2 Component Testing . . . 26
14.3 E2E Testing . . . 26

15 Architectural Strengths 27

16 Limitations and Future Enhancements 27
16.1 Current Limitations . . . 27
16.2 Recommended Enhancements . . . 28

17 Conclusion 28

1. Introduction and Problem Statement

1.1 Educational Challenges in GATE DA Preparation

The GATE (Graduate Aptitude Test in Engineering) Data Analytics examination is among India's most competitive postgraduate entrance examinations. Aspirants face several critical challenges:

1. **Content Vastness**: The GATE DA syllabus spans mathematics, computer science fundamentals, probability, statistics, linear algebra, machine learning, and artificial intelligence—requiring diverse subject expertise.
2. **Adaptive Learning Gap**: Traditional mock test portals provide static question sequences without personalizing difficulty or topic selection based on learner performance patterns.
3. **Lack of Real-time Guidance**: Students complete tests in isolation without immediate, explanatory feedback or next-step recommendations tailored to their performance profile.
4. **Teacher Engagement Inefficiency**: Educators lack systematic tools to identify struggling students, recommend targeted interventions, or track class-wide performance trends in real-time.
5. **Performance Prediction Absence**: Students cannot reliably estimate their likely GATE score based on mock performance, hindering time allocation and preparation intensity decisions.
6. **Incomplete Telemetry**: Existing platforms record only final scores, not the detailed event stream needed for explainability, remediation tracking, or longitudinal analytics.

1.2 Platform Vision

GATEWay addresses these challenges by providing:

- **Adaptive graph-based sequencing** that learns from each student's performance and dynamically adjusts question difficulty and topic selection.
- **Multi-engine recommendation portfolio** addressing next-best question selection, remediation paths, retry logic, suspicious rapid guessing detection, and personalized coaching.
- **Classroom orchestration tools** enabling teachers to author assignments, monitor submissions, identify at-risk students, and receive data-driven intervention suggestions.
- **Comprehensive telemetry** capturing both state (ELO, scores, progress) and events (actions, question answers, test completions) for rich observability.
- **Predictive scoring engine** that estimates likely GATE score bands based on learner proficiency factors, enabling informed preparation decisions.
- **Intelligent coaching system** using sophisticated rule-based logic to deliver personalized study recommendations and learning guidance.

The platform answers four fundamental educational questions:

$$\boxed{\begin{array}{ll} (1) & \text{What should the student solve next?} \\ (2) & \text{What should the student revise next?} \\ (3) & \text{Which learners need teacher intervention?} \\ (4) & \text{How can full learning paths be measured, reviewed, and explained?} \end{array}} \tag{1}$$

2. System Architecture and Design Philosophy

2.1 Architecture Overview

The platform is designed as a layered, multi-component ecosystem shown in Figure 1.

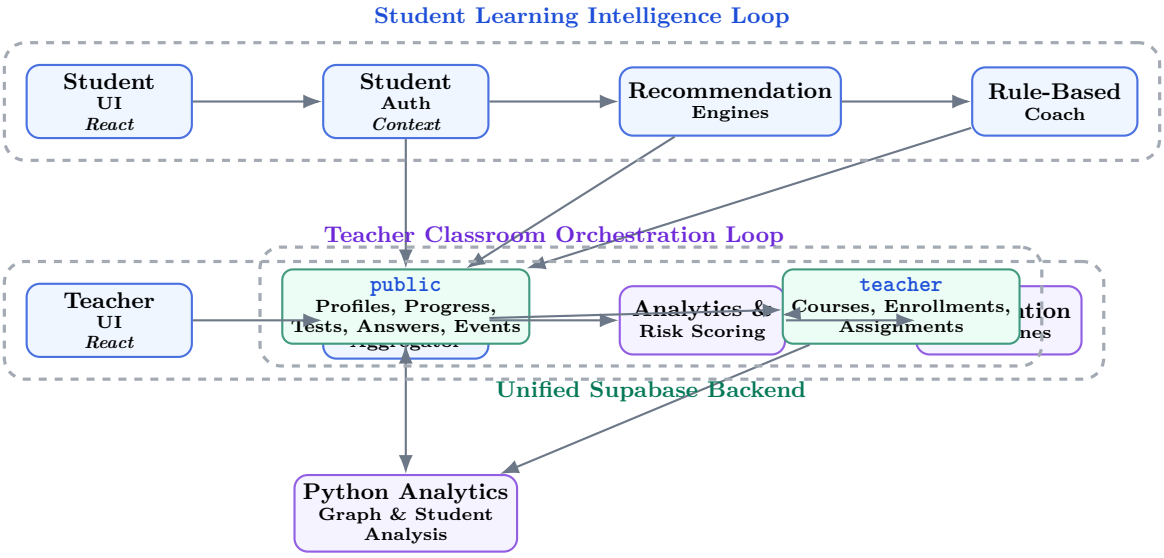

Figure 1: High-level system architecture. Student and teacher loops operate independently but converge on unified data schemas (public and teacher) stored in Supabase. Analytics notebooks enable exploratory research.

2.2 Technology Stack

Layer	Technologies
Frontend Framework	React 18 with TypeScript for type-safe UI development
Styling	Tailwind CSS with custom design tokens for consistent theming
UI Components	Radix UI and shadcn/ui for accessible, headless components
Routing	React Router v6 for client-side routing
Icons	Lucide Icons for vector graphics and consistent iconography
State Management	React Context API with custom hooks for domain-specific state
Data Fetching	TanStack React Query for server state and caching
Backend	Supabase (PostgreSQL, Auth, REST API, Realtime, Edge Functions)
Database Schema	Single project with `public` and `teacher` schemas
Row-Level Security	Supabase RLS policies ensuring multi-tenancy isolation
Charting	Recharts for in-app visualizations, PGFPlots for research outputs
Coaching System	Deterministic rule-based coaching engine with custom heuristics
Testing	Vitest for unit tests, Testing Library for components, Playwright for E2E
Build Tools	Vite for fast bundling and development
Research	Python (NetworkX, Pandas, NumPy, Matplotlib) for analytics notebooks

Table 1: Comprehensive technology stack across all layers.

2.3 Multi-Schema Data Modeling Philosophy

The architecture employs a single Supabase project with two exposed schemas, representing a critical design decision:

- **Public Schema**: Contains all student learning telemetry—profiles, progress, answered questions, test history, and activity events. Directly accessible to students, with teacher read access via row-level security.

- **Teacher Schema**: Contains classroom operations—teacher accounts, courses, student enrollments, assignments, and submissions. Isolated to teachers, with RLS preventing unauthorized access.

This separation:

- Maintains clear operational boundaries between student learning and teacher classroom management.
- Enables teacher analytics to traverse both schemas within row-level security constraints.
- Simplifies permission models by making the access pattern explicit.
- Supports efficient querying of class-level insights without mixing operational and learning data.

3. Data Model and Core Entities

3.1 Public Schema Tables

3.1.1 profiles

Stores authenticated user information, preferences, and account metadata.

Field	Type	Purpose
user_id	UUID	Foreign key to Supabase Auth
full_name	Text	Display name
email	Text	Contact information
elo	Integer	Current ELO rating (starts at 1600)
tier	Text	Proficiency tier (e.g., Bronze, Silver, Gold, Platinum)
theme	Text	User theme preference (light, dark)
created_at	Timestamp	Account creation date
updated_at	Timestamp	Last profile update

Table 2: Profile table structure for storing user accounts and preferences.

3.1.2 user_progress

Tracks subject-level performance metrics for adaptive sequencing.

Field	Type	Purpose
user_id	UUID	Student reference
subject	Text	Subject name (e.g., Linear Algebra, DBMS)
accuracy	Real	Fraction of correct answers in subject
questions_answered	Integer	Total questions attempted
weak_topic	Text	Identified weak area for remediation
updated_at	Timestamp	Last update

Table 3: User progress table tracking subject-level accuracy and weak areas.

3.1.3 answered_questions

Records every question answer with full context for replay and analysis.

Field	Type	Purpose
id	UUID	Record identifier
user_id	UUID	Student reference
question_id	Text	Question identifier from bank
submitted_answer	Text	Student's selected or written answer
is_correct	Boolean	Correctness flag
time_spent	Integer	Seconds elapsed (for rapid-guess detection)
attempt_number	Integer	Attempt count for retry tracking
created_at	Timestamp	Answer submission time

Table 4: Answered questions table for complete answer history.

3.1.4 test_history

Aggregated test completion records with optional replay metadata.

Field	Type	Purpose
id	UUID	Record identifier
user_id	UUID	Student reference
test_type	Text	Type (e.g., full_mock, topic_wise, adaptive)
test_name	Text	Human-readable test name
score	Integer	Raw score obtained
accuracy	Real	Fraction correct
completion_time	Integer	Total minutes spent
review_payload	JSON	Replayable data: question IDs, answers, metadata
created_at	Timestamp	Test completion time

Table 5: Test history table with optional review payload for explainability.

3.1.5 activity_events

Event stream capturing all significant user actions for observability and analytics.

Field	Type	Purpose
id	UUID	Record identifier
user_id	UUID	Student reference
event_type	Text	Event category (e.g., question_answered, test_completed)
metadata	JSON	Event-specific context
created_at	Timestamp	Event occurrence time

Table 6: Activity events table for comprehensive telemetry and analytics.

3.2 Teacher Schema Tables

3.2.1 teachers

Stores teacher account information and workspace metadata.

3.2.2 courses

Contains course information with unique join codes for student enrollment.

Field	Type	Purpose
id	UUID	Unique course identifier
teacher_id	UUID	Course owner reference
name	Text	Course display name
join_code	Text	Unique enrollment code
description	Text	Course description
created_at	Timestamp	Course creation date

Table 7: Courses table for classroom creation and management.

3.2.3 enrollments

Maps students to courses (many-to-many relationship).

3.2.4 assignments

Teacher-authored assignments with metadata and publication status.

3.2.5 submissions

Student assignment submissions with grades and feedback.

4. Student Learning Flow and Experience

4.1 Onboarding and Authentication

The student user flow begins with authentication via Supabase Auth:

1. Student signs up with email and password.
2. Supabase Auth creates a user account; a `profiles` record is automatically instantiated with default ELO (1600) and tier.
3. Student logs in; the `StudentAuthContext` restores local and cloud state via React Query.

4.2 Enrollment and Course Joining

Students join teacher classrooms via join codes:

1. Teacher creates a course and receives a unique 6-character join code.
2. Student enters the code in the platform's "Join Course" modal.
3. An `enrollment` record is created, linking the student to the course.
4. From that point, student progress is visible to the teacher via the class roster.

4.3 Learning Modes

Students can engage with the platform through four primary learning pathways:

4.3.1 Full Mock Test Mode

Students attempt complete GATE DA mock papers covering all topics in a single, timed session (typically 3 hours).

Workflow:

- Student selects a mock paper from available options.
- The system randomizes question order within the paper.
- Timer enforces time limits and provides progress feedback.
- Upon completion, the system computes raw score, accuracy, and test metadata.
- Results are stored in `test_history` with a `review_payload` enabling answer replay.

4.3.2 Topic-Wise Practice Mode

Students practice questions restricted to a single topic (e.g., "Graph Algorithms," "Probability").

Workflow:

- Student selects a subject and topic.
- System retrieves all questions for that topic (ordered by ELO or randomized).
- Student solves questions one-by-one; each submission updates the `answered_questions` table.
- Performance within the topic updates the `user_progress` row for that subject.

4.3.3 Adaptive Practice Mode

The system uses the graph-based adaptive engine to recommend the next best question based on learner state.

Workflow:

- Student initializes an adaptive session.
- The **adaptive next-best question engine** queries the student's current ELO, momentum, weak topics, and answered question set.
- The engine navigates the question recommendation graph (see Section 5) and selects the next question with reasoning.
- Student submits their answer; ELO, subject accuracy, and answered questions are updated.
- The system computes the next recommendation and repeats until the student ends the session.

4.3.4 Assignment Mode

Teachers publish assignments; students access and submit responses.

Workflow:

- Teacher authors and publishes an assignment within a course.
- Students enrolled in that course see the assignment in their assignment list.
- Student attempts the assignment (typically a curated set of questions with time limits).
- Submission is recorded in the `submissions` table; teacher can grade and provide feedback.

4.4 Performance Tracking and Metrics

As students engage, the system tracks:

- **ELO Rating**: Updated after each question based on expected vs. actual performance (Elo chess rating system adapted for accuracy).
- **Subject Accuracy**: Fraction of correct answers within each subject (e.g., linear algebra accuracy = 74%).
- **Weak Topics**: Topics with below-threshold accuracy, flagged for remediation.
- **Test History**: Aggregated results from full mocks, topic-wise sessions, and adaptive practice.
- **Momentum**: A label (hot, steady, cold) computed from recent correctness streak.
- **Completion Rate**: Fraction of all available questions answered (used in score prediction unlock criteria).

5. Adaptive Graph-Based Recommendation Engine

5.1 Design Objectives and Philosophy

The adaptive engine is the intellectual core of GATEWay. Rather than employing simple difficulty-based sorting, it:

- Constructs a semantic recommendation graph over the entire question bank.
- Models learner state as a multi-dimensional vector: current ELO, momentum, weak topics, recently attempted questions, and graph position.
- Navigates the graph using a hybrid scoring function combining heuristic rewards and graph-structure bonuses.
- Provides explainable recommendations with natural-language reasoning.
- Supports both standard adaptive sequences and targeted remediation after mistakes.

5.2 Question Graph Construction

The recommendation graph $G = (V, E)$ consists of:

- **Nodes (V)**: Question bank entries, each with attributes (subject, topic, ELO, marks, expected duration).
- **Edges (E)**: Directed edges representing pedagogical relationships.

5.2.1 Edge Types and Semantics

1. **Same-Topic Edges** (`same-topic`): Connect questions within the same subject and topic.
   - Useful for focused practice on a single skill.
   - Weight reflects difficulty progression: easier → harder or similar difficulty hops.
2. **Subject-Flow Edges** (`subject-flow`): Connect topics within the same subject in logical progression.
   - Example: Linear Algebra 101 → Mathematical Properties → Eigenvalues and Eigenvectors.
   - Support topic-wise sequential learning.
3. **Subject-Bridge Edges** (`subject-bridge`): Link related topics across different subjects.

- Example: Probability (Statistics) $\leftrightarrow$ Hypothesis Testing (Machine Learning).
- Enable cross-domain skill transfer and integrated learning paths.

Edge weights encode:

$$w(u, v) = \alpha_1 \cdot d_{\text{ELO}} + \alpha_2 \cdot d_{\text{marks}} + \alpha_3 \cdot d_{\text{topic-proximity}} + \alpha_4 \cdot d_{\text{type-similarity}} \tag{2}$$

where d_* denotes normalized distances (lower is better), and α_i are empirically tuned weights.

5.3 Learner State Representation

At any point in an adaptive session, the learner state is represented as a tuple:

$$\mathbf{s} = (\text{ELO}, \text{momentum}, \text{weak_topics}, Q_{\text{answered}}, Q_{\text{served}}, \text{remediation_progress}) \tag{3}$$

where:

- $\text{ELO} \in [0, 3000]$: Current rating (higher = stronger).
- $\text{momentum} \in \{\text{hot}, \text{steady}, \text{cold}\}$: Recent performance trend.
- $\text{weak_topics} \subseteq T$: Set of topics with accuracy $< \tau_{\text{weak}}$ (typically 60%).
- $Q_{\text{answered}} \subseteq V$: Set of questions already answered (never served again in a session).
- $Q_{\text{served}} \subseteq V$: Set of questions previously served (but not yet answered).
- remediation_progress: Metadata tracking remediation attempts for unresolved misses.

5.4 Target Policy Computation

The engine computes a target ELO and target difficulty:

5.4.1 Target ELO

$$\text{ELO}_{\text{target}} = \begin{cases} \text{ELO} + 100 & \text{if momentum} = \text{hot and accuracy} \geq 0.8 \\ \text{ELO} + 50 & \text{if momentum} = \text{steady and accuracy} \geq 0.7 \\ \text{ELO} & \text{otherwise (steady or declining)} \\ \text{ELO} - 100 & \text{if momentum} = \text{cold and accuracy} < 0.5 \end{cases} \tag{4}$$

Intuitively: students on hot streaks are challenged with harder questions; struggling students receive easier review material.

5.4.2 Target Difficulty

$$\text{difficulty}_{\text{target}} = \text{map_elo_to_difficulty}(\text{ELO}_{\text{target}}) \tag{5}$$

where the mapping is typically:

- $\text{ELO} < 1600$: Easy
- $1600 \leq \text{ELO} < 2000$: Medium
- $2000 \leq \text{ELO} < 2500$: Hard
- $\text{ELO} \geq 2500$: Expert

5.5 Candidate Selection and Neighborhood Navigation

Given the learner state and target policy, the engine branches on whether an unresolved miss exists:

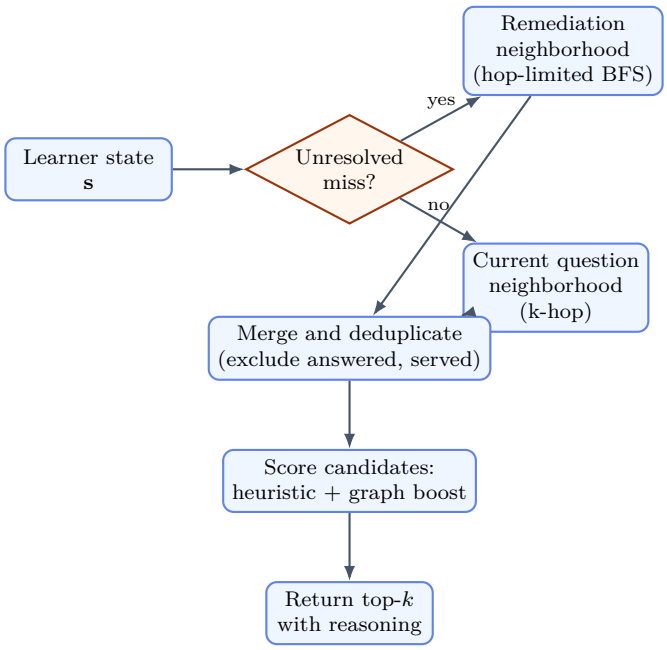

Figure 2: Adaptive recommendation pipeline decision logic.

5.5.1 Remediation Neighborhood

When a student answers a question incorrectly (unresolved miss), the engine defines a remediation neighborhood:

$$N_{\text{remed}}(q, h) = \{q' \in V : d_G(q, q') \leq h, \text{ not answered }, \text{ difficulty}(q') < \text{difficulty}(q)\} \tag{6}$$

where $d_G(q, q')$ is the graph distance (hops), and h (typically 2–3) is the maximum hop limit. This ensures:

- Questions are related to the originally missed question (nearby in graph).
- But easier (to build confidence before retry).
- And haven't been answered before.

5.5.2 Current Question Neighborhood

In standard (non-remediation) adaptive flow:

$$N_{\text{current}}(q, k) = \{q' \in V : d_G(q, q') \leq k, \text{ not answered }, \text{ not served }\} \tag{7}$$

Typical $k = 2$ (2-hop neighbors).

5.6 Candidate Scoring Function

After assembling the candidate pool C, the engine scores each candidate $q' \in C$:

$$\text{score}(q') = w_h \cdot h(q') + w_g \cdot g(q') \tag{8}$$

where:

- $h(q')$ is a **heuristic score** combining:
  1. Distance to target difficulty: $d_{\text{diff}} = |\text{difficulty}(q') - \text{difficulty}_{\text{target}}|$
  2. Distance to target ELO: $d_{\text{elo}} = |\text{ELO}(q') - \text{ELO}_{\text{target}}|$
  3. Weak-topic bonus: if q' is in a weak topic, add bonus β_{weak}.
  4. Type diversity: if q' type differs from the current question, add bonus β_{type}.

  Typically:

$$h(q') = 1.0 - \frac{w_1 \cdot d_{\text{diff}} + w_2 \cdot d_{\text{elo}}}{C_{\text{norm}}} + \beta_{\text{weak}} \cdot \mathbb{1}[q' \in \text{weak_topics}] + \beta_{\text{type}} \cdot \mathbb{1}[\text{type}(q') \neq \text{type}(q)] \quad (9)$$

- $g(q')$ is a **graph boost** rewarding structural properties:

$$g(q') = \gamma_1 \cdot \rho_{\text{in}}(q') + \gamma_2 \cdot \rho_{\text{out}}(q') + \gamma_3 \cdot c(q') \quad (10)$$

  where:

  – $\rho_{\text{in}}(q')$ = normalized in-degree (how many questions lead to q').

  – $\rho_{\text{out}}(q')$ = normalized out-degree (how many questions q' leads to).

  – $c(q')$ = normalized centrality measure (e.g., betweenness centrality).

  Nodes with high centrality are pedagogically important ẅhub q̈uestions, reflecting well-established learning pathways.

Weights w_h, w_g, and coefficients are tuned empirically via A/B testing or offline replay.

5.7 Output and Explainability

The engine returns the top-k (typically $k = 1$) candidate with:

- Question ID and full metadata (text, options, marks, expected difficulty).
- Natural-language explanation: e.g., "Continuing on Linear Algebra—the next question is harder to strengthen your eigenvalue skills."
- Graph metadata: hop distance, centrality signal, neighborhood type (remediation vs. standard).
- Target ELO and expected difficulty for transparency.

6. Remediation and Retry Engine

When a student submits an incorrect answer, the system marks the question as an unresolved miss. The remediation engine manages the recovery process:

6.1 Remediation Workflow

$$\boxed{\begin{array}{ll} (1) & \text{Flag question as unresolved miss} \\ (2) & \text{Serve remediation neighbors (easier, related)} \\ (3) & \text{Track accuracy and step count within remediation sequence} \\ (4) & \text{After } N_{\text{remed}} \text{ successful steps or accuracy} \geq A_{\text{remed}}: \text{ unlock retry of original} \\ (5) & \text{If retry is correct: clear miss, mark as resolved} \\ (6) & \text{If retry is incorrect: re-queue for remediation} \end{array}} \quad (11)$$

6.2 Remediation Exit Criteria

Remediation concludes when:

$$\text{steps_taken} \geq N_{\text{remed}} \quad \wedge \quad \text{remediation_accuracy} \geq A_{\text{remed}} \tag{12}$$

Typical values: $N_{\text{remed}} = 3$ remediation steps, $A_{\text{remed}} = 0.75$ (75% accuracy).

If these criteria are not met within a session, the miss carries forward; the retry is queued for the next session.

7. Practice Review and Rapid-Guess Detection Engine

7.1 Motivation: Quality of Evidence

Not all correct answers represent equal mastery. A student might answer a complex multi-step problem correctly but in 10 seconds—suspicious and likely guessing. The rapid-guess detection engine in `src/lib/practiceReview.ts` flags such scenarios.

7.2 Threshold Computation

For each question, the engine estimates a base minimum solve time:

$$t_{\min}(q) = \alpha_0 + \alpha_1 \cdot \text{stem_length}(q) + \alpha_2 \cdot \text{options_length}(q) + \alpha_3 \cdot \text{complexity_index}(q) \tag{13}$$

where complexity index is higher for questions labeled as multi-step or requiring computation.

Typical coefficient values (empirically tuned):

- $\alpha_0 = 10$ seconds (baseline reading time).
- $\alpha_1 = 0.02$ seconds per character in stem.
- $\alpha_2 = 0.01$ seconds per character in option set.
- $\alpha_3 = 30$ seconds for high-complexity questions.

7.3 Penalty Application

If $t_{\text{spent}}(q) < t_{\min}(q)$:

$$\text{ELO_penalty} = (1 - \frac{t_{\text{spent}}}{t_{\min}}) \cdot 20 \text{ points} \tag{14}$$

$$\text{ELO_gain}(q) = \max(0, \text{standard_gain} - \text{ELO_penalty}) \tag{15}$$

The student still receives marks, but their ELO gain is reduced, preventing rapid guessing from inflating proficiency ratings.

8. Coach Engine

8.1 Purpose and Scope

The coach provides personalized study recommendations, learning plans, and guidance to students. Rather than being a chatbot, it focuses on structured, actionable coaching:

- Identifies weak and strong subject areas.
- Recommends topic prioritization and revision strategies.
- Generates same-day, 3-day, and 7-day study plans.
- Provides mock preparation strategies and effort allocation advice.
- Responds to student questions with carefully crafted rule-based recommendations.

8.2 Rule-Based Implementation Architecture

The coach is built on a pure rule-based heuristic engine with no external dependencies. All coaching logic is hand-crafted and parameterized based on empirical observations from GATE DA preparation patterns.

Key Properties:

1. **Fast Execution**: Response generation completes in milliseconds, ensuring responsive user experience.
2. **Deterministic and Explainable**: Every recommendation follows transparent, understandable rules that can be audited and explained to students.
3. **No External Dependencies**: The coach operates offline and does not require API calls, network connectivity, or external services.
4. **Custom Tuning**: Parameters are adjusted based on observed student behavior patterns within the platform.

8.3 Coach Input State

The coach receives:

$$\mathbf{c} = (\text{ELO}, \text{tier}, \text{overall_acc}, \text{weak_subjects}, \text{strong_subjects}, \text{streak}, \text{recent_tests}, \text{focus_area}) \tag{16}$$

where:

- $\text{ELO} \in [0, 3000]$: Current proficiency.
- tier: Categorical proficiency level (Bronze, Silver, Gold, Platinum).
- $\text{overall_acc} \in [0, 1]$: Fraction of all questions answered correctly.
- weak_subjects: Set of subjects with accuracy $< \tau_{\text{weak}}$.
- strong_subjects: Set of subjects with accuracy $\geq \tau_{\text{strong}}$.
- streak: Current correct-answer streak length.
- recent_tests: List of recent test results (score, accuracy, time).
- focus_area: Student-specified topic of interest or concern.

8.4 Deterministic Coach Logic

The fallback coach implements heuristic rules:

8.4.1 Daily Study Suggestion

$$\text{suggestion} = \begin{cases} \text{“Focus on } w_1 \text{ today—you have } (100 - \text{acc}_{w_1})\% \text{ room for improvement.”} & \text{if weak_subjects not empty} \\ \text{“Great momentum! Tackle harder problems in } s_1 \text{ today.”} & \text{if streak} > 5 \\ \text{“Review fundamentals in } w_1 \text{ to build confidence.”} & \text{otherwise} \end{cases} \tag{17}$$

8.4.2 7-Day Study Plan

The coach allocates effort across weak, strong, and new topics:

$$\boxed{\begin{array}{l} \text{Day 1-2 : Weak topic 1} \quad (40\% \text{ effort}) \\ \text{Day 3-4 : Weak topic 2} \quad (30\% \text{ effort}) \\ \text{Day 5-6 : Strong topic (mock)} \quad (20\% \text{ effort}) \\ \text{Day 7 : Revision or new topic} \quad (10\% \text{ effort}) \end{array}} \tag{18}$$

Effort allocation is parameterized by tier and recent performance.

8.5 Coach Reasoning Logic

The rule-based engine employs a multi-step pipeline to generate recommendations:

8.5.1 Step 1: Student Profile Analysis

The system analyzes the student input state to compute:

- Proficiency level based on current ELO (low: <1800, medium: 1800-2200, high: >2200).
- Performance stability (comparing recent vs. historical accuracy).
- Effort allocation efficiency (questions per topic studied).
- Motivation signal (streak length and session frequency).

8.5.2 Step 2: Weakness Severity Ranking

Weak subjects are ranked by severity:

$$\text{severity}(s) = (1 - \text{accuracy}(s)) \times \text{weight}(s) \tag{19}$$

where weight reflects importance in GATE DA (e.g., probability gets higher weight than niche topics).

8.5.3 Step 3: Personalized Recommendations

Based on the student profile, the coach selects from a curated set of advice templates:

- If ELO < 1800 and accuracy $< 50\%$: Focus on fundamentals and concept clarity.
- If ELO $\in [1800, 2200]$ and improving: Maintain momentum by tackling harder questions.
- If ELO > 2200 and accuracy $> 75\%$: Shift focus to speed optimization and mock practice.
- If declining trend detected: Recommend topic-specific review and reduced difficulty.

8.5.4 Step 4: Study Plan Generation

The 7-day plan is constructed by:

- Day 1-2: Highest-severity weak topic (primary focus).
- Day 3: Second-highest-severity weak topic and foundational concepts.
- Day 4-5: Integrated practice covering multiple topics.
- Day 6: Mock test or full-paper attempt to assess integrated readiness.
- Day 7: Targeted revision of mistakes from the mock test.

Each day's plan includes specific exercise counts, difficulty targets, and expected study duration.

9. Teacher Analytics and Intervention Intelligence

9.1 Teacher Workspace Aggregation

The teacher workspace hook (in `hooks/useTeacherWorkspace.ts`) aggregates classroom data:

1. Fetch all teaching credentials for the logged-in teacher.
2. Query all courses created by the teacher.
3. For each course, fetch enrollments, assignments, and submissions.
4. Execute a joined query over `public` and `teacher` schemas to retrieve student profiles, progress, test history, and activity events for enrolled learners.

This creates a unified view of classroom state available to the teacher dashboard.

9.2 Teacher Summary Computations

9.2.1 Student Summaries

For each enrolled student, compute:

Metric	Definition
Overall Accuracy	$\text{acc} = \frac{\#\text{ correct answers}}{\#\text{ total answered}}$
ELO	Current rating from `profiles`
Weak Topics (top 3)	Subjects with accuracy < 0.6, sorted by severity
Assignment Completion	Fraction of assigned questions submitted
Risk Level	Composite score (see Section 9.3)
Recent Activity	Last action timestamp (question answered, test completed, etc.)
Performance Trend	7-day accuracy comparison

Table 8: Student summary metrics computed for teacher viewing.

9.2.2 Course Summaries

For each course, compute:

- Enrollment count: number of enrolled students.
- Average accuracy: mean of all student accuracies.
- Completion rate: fraction of assigned questions completed.

- Weak topics (class-wide): frequency of below-threshold accuracy by subject.
- At-risk student count: students with risk level $> T_{\text{risk}}$ (typically 0.7).
- Top performers: students with accuracy $> T_{\text{top}}$ (typically 0.8).

9.3 Risk Scoring

Risk distills learner challenges into a single actionable score. The composite risk score is:

$$\text{Risk}(s) = w_1 \cdot r_{\text{acc}}(s) + w_2 \cdot r_{\text{comp}}(s) + w_3 \cdot r_{\text{elo}}(s) \tag{20}$$

where:

- $r_{\text{acc}}(s) = 1 - \text{accuracy}(s)$: Accuracy-based risk (lower accuracy = higher risk).
- $r_{\text{comp}}(s) = 1 - \text{completion_rate}(s)$: Completion-based risk (not engaging enough).
- $r_{\text{elo}}(s) = \begin{cases} 1 & \text{if ELO}(s) < 1800 \\ 0 & \text{otherwise} \end{cases}$: ELO readiness (students under threshold need support).

Typical weights: $w_1 = 0.5$, $w_2 = 0.3$, $w_3 = 0.2$ (accuracy is the primary risk signal).

Students with Risk ≥ 0.7 appear in the teacher's intervention queue.

9.4 Intervention Recommendations

For each at-risk student, the teacher analytics module recommends actions:

1. **Identify the top weak topic** with lowest accuracy.
2. **Suggest a targeted assignment**: author and publish an exercise within that weak topic.
3. **Check remediation progress**: query `answered_questions` to see if the student is successfully remediating past misses.
4. **Schedule a follow-up**: recommend a check-in date based on the assignment deadline.

Example recommendation output:

```
Recommendation: Priya needs help with Probability & Statistics.
- Current accuracy: 45%
- Recent miss: Q2834 (binomial distribution)
- Suggested action: Publish a 5-question probability assignment
- Follow-up: Check progress in 2 days
```

10. GATE Score Prediction Engine

10.1 Vision and Purpose

The GATE score prediction module `src/components/GateScorePrediction.tsx` addresses a critical learner need: estimating likely final GATE scores based on current preparation progress. This enables students to:

- Set realistic targets (e.g., target 85+ percentile rank).
- Allocate remaining study time effectively.

- Identify whether their preparation pace is sufficient.
- Build confidence through informed end-state projections.

10.2 Unlock Criteria

The score predictor is deliberately kept exclusive to avoid premature or unreliable predictions. It unlocks only when:

$$\text{Predictor UNLOCKED} \iff \begin{cases} \text{ELO} \geq 2500 \quad \text{AND} \\ \text{full_mock_completion} \geq 0.70 \quad \text{AND} \\ \text{topic_wise_completion} \geq 0.70 \quad \text{AND} \\ \text{adaptive_completion} \geq 0.70 \end{cases} \tag{21}$$

This ensures the model operates on substantial evidence:

- ELO 2500+ indicates solid proficiency.
- 70% completion across all three learning modes ensures broad coverage and engagement depth.

10.3 Core Formula: Predicted Score Basis

Once unlocked, the predictor generates a **score basis**—a normalized composite score from four key dimensions:

$$\text{ScoreBasis} = w_1 \cdot \text{Acc}_{\text{norm}} + w_2 \cdot \text{ELO}_{\text{norm}} + w_3 \cdot \text{Cons}_{\text{norm}} + w_4 \cdot \text{Impr}_{\text{norm}} \tag{22}$$

where the weights are: $w_1 = 0.40$, $w_2 = 0.30$, $w_3 = 0.20$, $w_4 = 0.10$.

Below, we explain each component in detail.

10.3.1 Component 1: Overall Accuracy (Acc)

Conceptual Definition:

$$\text{Accuracy} = \frac{\#\text{ questions answered correctly}}{\#\text{ total questions answered}} \tag{23}$$

Variable Definitions:

- C: Count of correct answers (from `answered_questions` where `is_correct`=true).
- T: Total questions answered.
- $\text{Accuracy}_{raw} = \frac{C}{T}$.

Normalization:

$$\text{Acc}_{\text{norm}} = \min\left(1.0, \frac{\text{Accuracy}_{raw} - 0.5}{0.35}\right) \tag{24}$$

This maps raw accuracy to $[0, 1]$:

- Accuracy $\leq 50\%$ (struggling) $\rightarrow$ normalized score ≤ 0.
- Accuracy $= 85\%$ (strong) $\rightarrow$ normalized score ≈ 1.0.
- Accuracy $> 85\%$ $\rightarrow$ capped at 1.0.

The lower bound (50%) reflects that random guessing on MCQC with 4 options yields 25% accuracy; 50% is a meaningful threshold above guessing.

10.3.2 Component 2: ELO Rating (ELO_{norm})

Conceptual Definition: The ELO rating (from `profiles`) reflects learner proficiency through wins and losses relative to question difficulty. Higher ELO = stronger learner.

Variable Definitions:

- $\text{ELO}_{current}$: Current ELO from user profile (range: 0—3000).
- $\text{ELO}_{baseline} = 1600$: Starting ELO.
- $\text{ELO}_{max} = 2800$: Typical maximum reached by very strong learners.

Normalization:

$$\text{ELO}_{\text{norm}} = \frac{\text{ELO}_{current} - \text{ELO}_{baseline}}{\text{ELO}_{max} - \text{ELO}_{baseline}} = \frac{\text{ELO}_{current} - 1600}{1200} \tag{25}$$

This maps:

- ELO = 1600 (baseline) $\rightarrow$ 0.0.
- ELO = 2400 (strong) $\rightarrow$ 0.667.
- ELO = 2800 (very strong) $\rightarrow$ 1.0.

10.3.3 Component 3: Subject Consistency ($\text{Cons}_{\text{norm}}$)

Conceptual Definition: A learner with wildly varying subject accuracies (e.g., 90% in one subject, 40% in another) is less prepared than one with balanced proficiency. Consistency measures how evenly performance is distributed across subjects.

Variable Definitions:

- $S = \{s_1, s_2, \ldots, s_n\}$: Set of subjects (e.g., Linear Algebra, DBMS, Probability, ML, AI).
- acc_i = accuracy in subject s_i (from `user_progress`).
- $\bar{\text{acc}} = \frac{1}{n}\sum_{i=1}^{n} \text{acc}_i$: Mean accuracy across subjects.

Consistency Metric (Inverse Coefficient of Variation):

$$\text{Consistency}_{raw} = 1 - \frac{\sigma(\text{acc}_1, \ldots, \text{acc}_n)}{\bar{\text{acc}} + \epsilon} \tag{26}$$

where σ is standard deviation and ϵ is a small value (e.g., 0.01) to avoid division by zero.

Intuitively:

- If all subjects have accuracy 75%: $\sigma = 0 \Rightarrow$ Consistency = 1.0 (perfect).
- If accuracies are [90%, 40%, 70%, 65%, 60%], σ is large $\Rightarrow$ Consistency is lower.

Normalization:

$$\text{Cons}_{\text{norm}} = \max(0, \text{Consistency}_{raw}) \tag{27}$$

Bounded to $[0, 1]$.

Interpretation: A student with 80% overall accuracy will score higher if that 80% is evenly distributed (75%, 85%, 78%, 82%) vs. skewed (95%, 45%, 80%, 85%). The former shows better mastery of breadth; the latter shows gaps that may cause score loss in the actual exam.

10.3.4 Component 4: Improvement Factor ($\text{Impr}_{\text{norm}}$)

Conceptual Definition: Students showing upward performance trends (improving over time) are likely to maintain or extend that improvement. The improvement factor rewards recent accelerating progress.

Variable Definitions:

- $T_{\text{recent}} \in (0, T_{\text{all}}]$: Recent time window (e.g., last 14 days).
- $\text{acc}_{\text{recent}}$: Accuracy in recent window.
- $\text{acc}_{\text{prior}}$: Accuracy in all prior data.
- $\Delta\text{acc} = \text{acc}_{\text{recent}} - \text{acc}_{\text{prior}}$: Improvement magnitude.

Improvement Metric:

$$\text{Improvement}_{raw} = \begin{cases} \min(0.15, \Delta\text{acc}) & \text{if } \Delta\text{acc} > 0 \quad \text{(improving)} \\ \max(-0.15, \Delta\text{acc}) & \text{if } \Delta\text{acc} \leq 0 \quad \text{(declining)} \end{cases} \tag{28}$$

This captures improvements/declines up to ± 15 percentage points.

Normalization:

$$\text{Impr}_{\text{norm}} = \frac{\text{Improvement}_{raw} + 0.15}{0.30} \tag{29}$$

Maps to $[0, 1]$:

- $\Delta\text{acc} = -15\%$ (declining) $\rightarrow 0.0$.
- $\Delta\text{acc} = 0\%$ (stable) $\rightarrow 0.5$.
- $\Delta\text{acc} = +15\%$ (improving) $\rightarrow 1.0$.

Interpretation: A student improving by 5 percentage points recently is rewarded; one declining is penalized. This captures momentum.

10.4 Score Band Generation from Basis

Once ScoreBasis $\in [0, 1]$ is computed, the predictor generates GATE score estimates. The GATE DA exam scoring is typically:

- Total marks: 100 (occasionally 85 or 90 depending on exam).
- Question distribution: usually 55 MCQ (1 mark each) + 10 NAT (numerical) questions (various marks).

For simplicity, assume 100-mark scale.

10.4.1 Expected Score

$$\text{Expected_Score} = 40 + 50 \cdot \text{ScoreBasis} \tag{30}$$

This maps ScoreBasis to an expected GATE score:

- ScoreBasis = 0.0 (barely qualifying) $\rightarrow$ Expected = 40.
- ScoreBasis = 0.5 (moderate) $\rightarrow$ Expected = 65.
- ScoreBasis = 1.0 (excellent) $\rightarrow$ Expected = 90.

10.4.2 Minimum and Maximum Score Bands

To provide a confidence range, the model estimates bounds:

$$\text{Minimum_Score} = \text{Expected_Score} - 10 \cdot (1 - \text{ScoreBasis}) \tag{31}$$

$$\text{Maximum_Score} = \text{Expected_Score} + 10 \cdot \text{ScoreBasis} \tag{32}$$

Intuitively:

- High ScoreBasis (near 1.0) → tight band (high confidence).
- Low ScoreBasis (near 0.0) → wider band (more uncertainty).

Example:

- ScoreBasis = 0.8 ⇒ Expected = 80, Min = 72, Max = 88.
- ScoreBasis = 0.4 ⇒ Expected = 60, Min = 46, Max = 64.

10.4.3 Confidence Score

Confidence reflects certainty in the prediction:

$$\text{Confidence} = 0.5 + 0.4 \cdot \text{ScoreBasis} + 0.1 \cdot \text{Completion_Rate} \tag{33}$$

where Completion_Rate is the fraction of all GATE DA questions answered (0 to 1).

This maps to:

- Strong basis + high completion → Confidence near 1.0 (very confident).
- Weak basis + low completion → Confidence near 0.5 (moderate confidence).

10.4.4 Recommendation Text

The predictor generates contextual guidance:

$$\text{RecommendationText} = \begin{cases} \text{“Excellent! With your current preparation, expect GATE score 80–90.} \\ \text{Focus on maintaining consistency across all topics.”} & \text{if Expected} \geq 75 \\ \\ \text{“Good progress. Expected score 60–70. Strengthen weak topics:} \\ \text{[weak subjects list]. Increase mock attempt frequency.”} & \text{if } 60 \leq \text{Expected} < 7 \\ \\ \text{“Your preparation is on track for 40–60. Accelerate study} \\ \text{intensity, especially in: [weak subjects]. Try 1 mock/week.”} & \text{if Expected} < 60 \end{cases} \tag{34}$$

10.5 Worked Example

Assume a student with:

- Accuracy: 78%.
- ELO: 2350.

- Subject accuracies: [85%, 72%, 70%, 75%, 80%] (standard deviation: 6%).
- Mean: 76.4%.
- Consistency: $1.0 - \frac{0.06}{0.764+0.01} \approx 0.922$.
- Recent accuracy (last 14 days): 82%; all prior: 75%; $\Delta\text{acc} = 7\%$.

Normalization:

- $\text{Acc}_{\text{norm}} = \min(1.0, \frac{0.78-0.5}{0.35}) = \frac{0.28}{0.35} \approx 0.80$.
- $\text{ELO}_{\text{norm}} = \frac{2350-1600}{1200} \approx 0.625$.
- $\text{Cons}_{\text{norm}} = 0.922 \approx 0.92$.
- $\text{Impr}_{\text{norm}} = \frac{0.07+0.15}{0.30} \approx 0.73$.

Score Basis:

$$\text{ScoreBasis} = 0.40 \cdot 0.80 + 0.30 \cdot 0.625 + 0.20 \cdot 0.92 + 0.10 \cdot 0.73 \tag{35}$$

$$= 0.32 + 0.1875 + 0.184 + 0.073 = 0.7645 \tag{36}$$

Score Predictions:

- Expected: $40 + 50 \cdot 0.7645 = 40 + 38.225 = 78.2 \approx 78$.
- Minimum: $78 - 10 \cdot (1 - 0.7645) = 78 - 2.355 = 75.6 \approx 76$.
- Maximum: $78 + 10 \cdot 0.7645 = 78 + 7.645 = 85.6 \approx 86$.
- Confidence: $0.5 + 0.4 \cdot 0.7645 + 0.1 \cdot 0.70 \approx 0.5 + 0.306 + 0.07 = 0.876$ (high confidence).

Recommendation: "Good progress. Expected GATE score: 76–86 (best estimate: 78). Consistency is strong across topics. Recent acceleration (+7%) is encouraging—maintain this pace by continuing daily practice on weak areas."

11. Telemetry, Persistence, and Event-Driven Observability

11.1 State Persistence Strategy

The platform persists learner state in two forms:

11.1.1 Persistent State (Tables)

- ELO, tier, and profile metadata in `profiles`.
- Subject accuracy and weak-topic tracking in `user_progress`.
- Complete answer history in `answered_questions` (for replay and remediation).
- Test summaries with optional `review_payload` in `test_history`.

This state is queried by the recommendation engines, analytics modules, and student dashboards.

11.1.2 Event Stream (activity_events table)

In addition to state, every significant action is logged as an event:

Event Type	Metadata
`user_signed_up`	user_id, email, timestamp
`user_signed_in`	user_id, timestamp
`question_answered`	user_id, question_id, is_correct, time_spent
`test_started`	user_id, test_type, test_name
`test_completed`	user_id, test_type, score, accuracy
`graph_path_started`	user_id, session_id
`graph_path_completed`	user_id, session_id, questions_count, accuracy
`course_joined`	user_id, course_id
`course_left`	user_id, course_id
`assignment_published`	teacher_id, course_id, assignment_id
`submission_received`	student_id, assignment_id, score

Table 9: Sample activity events and associated metadata.

11.2 Why This Design Matters

The dual stream (state + events) is critical:

1. **Explainability**: Given a student's current ELO, the event stream enables reconstruction of how they arrived at that rating (what answers led to ELO changes).
2. **Observability**: Teachers and administrators can inspect recent activity, understand student engagement patterns, and identify behavioral anomalies.
3. **Research and Analytics**: The event stream enables longitudinal studies, cohort analysis, and identification of effective learning sequences.
4. **Audit Trail**: Every significant action is recorded, supporting academic integrity monitoring.
5. **Dashboard and History Views**: Students can replay past sessions using the `review_payload`, understanding which questions they missed and why.

12. History Review and Explainable Replaying

A salient feature is the **review_payload** field in `test_history`. When a test completes, the system optionally records:

- List of question IDs presented (in order).
- For each question: submitted answer, correctness, time spent, any rapid-guess warning.
- ELO change due to the question (if available).
- Explanation metadata (graph information, remediation notes).

This enables a **replayable history view**:

1. Student selects a past test from history.
2. System reconstructs the session question-by-question.
3. For each question, the app displays: question text, student's answer, correct answer, explanation.
4. Rapid-guess warnings are highlighted: "This was answered in 5 seconds, but estimated minimum was 12 seconds. ELO boost was reduced."

This transforms history from opaque score summaries into interactive learning artifacts.

13. Research Assets and Analytical Notebooks

The repository includes Python-based analysis tools:

- `generate_gate_da_question_graph_notebook.py`: Creates a Jupyter notebook analyzing the question recommendation graph (centrality, connectivity, degree distribution).
- `generate_gate_da_student_progress_notebook.py`: Generates a notebook with student cohort analytics (ELO distribution, accuracy trends, subject performance heatmaps).
- Teacher dashboard mockup generator: Creates sample visualizations showing class-wide metrics.

These notebooks use the same data model and provide exploratory analysis complementary to the live UI.

14. Testing and Quality Assurance

14.1 Test Coverage for Adaptive Engine

The adaptive engine is the highest-risk component, so it receives focused testing:

1. **Same-Topic Progression**: After answering a question correctly, the next recommendation should be in the same topic but harder (or similar difficulty).
2. **Cross-Topic Remediation**: After answering incorrectly, remediation recommendations should be easier and preferably in the same topic.
3. **Retry Logic**: After sufficient remediation with high accuracy, a retry of the missed question should be queued.
4. **Exclusion of Answered Questions**: Previously answered questions must never be served again in a session.
5. **ELO Bounds**: ELO changes must stay within expected ranges, and rapid-guess penalties must be correctly applied.

Tests are written in Vitest with fixtures for sample graphs and learner states.

14.2 Component Testing

React components (especially recommendation displays and adaptive flows) are tested using Testing Library to ensure:

- Recommendations are rendered with rationale text.
- Answer submission correctly updates state.
- History replay correctly reconstructs past sessions.
- Teacher dashboards correctly filter and display student data.

14.3 E2E Testing

Playwright tests cover end-to-end flows:

1. Student signup, login, and profile creation.
2. Joining a teacher course via code.
3. Attempting an adaptive session and verifying ELO updates.

4. Reviewing history and replaying answers.
5. Teacher viewing classroom dashboards and intervention recommendations.

15. Architectural Strengths

1. **Clean Role Separation**: Student and teacher experiences are conceptually distinct, reflected in separate UI paths and data schemas.
2. **Explainable Recommendations**: Every recommendation includes reasoning (graph hop distance, difficulty target, weak-topic signal), supporting learner trust and educator understanding.
3. **Comprehensive Telemetry**: The `activity_events` stream enables observability into learner behavior, supporting debugging, analytics, and curiosity-driven research.
4. **Intelligent Rule-Based Coaching**: The coaching engine is built on carefully designed heuristics and domain knowledge, ensuring transparent, understandable, and reliable recommendations without external dependencies.
5. **Replayable History**: The `review_payload` upgrade converts static score records into interactive learning artifacts for deeper reflection.
6. **Remediation-Aware Sequencing**: The adaptive engine doesn't just select harder questions; it proactively identifies unresolved misses and queues targeted recovery sequences.
7. **Teacher-Centric Actional Insights**: Analytics output prioritizes actionable recommendations (suggest specific assignments, identify at-risk students) over passive reporting.
8. **Thoughtful Score Prediction**: The GATE score predictor balances mathematical rigor (detailed formula with four normalized components) with practical utility (confidence bands, recommendation text).
9. **Research Layer**: Python notebooks and analysis tooling complement the live UI, supporting exploratory analytics and academic inquiry.

16. Limitations and Future Enhancements

16.1 Current Limitations

1. **Formal Experiment Tracking**: The platform lacks a first-class A/B testing framework. Recommendation policy changes are not yet systematically compared via randomized experiments.
2. **Longitudinal In-App Analytics**: While the `activity_events` stream is rich, fewer longitudinal visualizations are surfaced directly in the app. Most longitudinal analysis happens in notebooks.
3. **Score Prediction History**: Current predictions are point-in-time. Persisting historical predictions would enable trend analysis ("How did my predicted score change over the past month?").
4. **Spaced Repetition**: The remediation engine is reactive (triggered after misses). A proactive spaced-repetition system would queue forgotten topics for revisiting.
5. **Prerequisite Graphs**: The current graph is similarity-based. An explicit prerequisite graph (e.g., Linear Algebra Fundamentals → Eigenvalues) would improve sequencing.
6. **Cross-Subject Integration**: While the graph includes subject-bridge edges, deeper integrative questions (combining multiple subjects) are underrepresented.

16.2 Recommended Enhancements

1. **A/B Testing Framework**: Implement experiment tracking so that recommendation engine variants can be systematically compared on engagement, learning gain, and score prediction accuracy.
2. **Spaced Repetition Scheduler**: Integrate a Leitner-box style system for proactive weak-topic revisiting at optimal intervals.
3. **Prerequisite Awareness**: Explicitly model prerequisites and enforce sequencing (don't serve eigenvalue questions until linear-solve fundamentals are mastered).
4. **In-App Longitudinal Dashboards**: Display student ELO trends, weak-topic evolution, and predicted score trajectory directly in student and teacher dashboards.
5. **Adaptive Difficulty Calibration**: Use item-response theory (IRT) to calibrate question difficulties and adapt them based on population performance.
6. **Peer Benchmarking**: Compare student performance against class cohort (anonymous) to provide contextualized feedback.
7. **Intelligent Content Recommendation**: Beyond questions, recommend:
   - Reading materials or videos for weak topics.
   - Worked examples and step-by-step solutions.
   - Similar previously-solved questions for reinforcement.
8. **Real-Time Collaboration Features**: Enable peer study groups, shared notes, and instructor office hours within the platform.

17. Conclusion

GATEWay is a comprehensive, multi-dimensional educational intelligence platform that transcends simple mock-test portals. Its core value lies not in any single component, but in the seamless integration of:

- **Graph-based adaptive sequencing** that personalizes question difficulty and topic selection.
- **Multiple specialized recommendation engines** addressing next-best questions, remediation, rapid-guess detection, and personalized coaching.
- **Classroom orchestration tools** enabling teachers to author assignments, monitor progress, and receive intervention recommendations.
- **Comprehensive telemetry** capturing both persistent state (ELO, scores) and event streams (actions, decisions) for deep observability.
- **Predictive scoring** powered by a rigorous, four-component formula that estimates likely GATE scores with confidence bands.
- **Rule-based coaching system** that delivers personalized study recommendations and guidance based on student performance analysis.
- **Explainable, replayable history** enabling students to revisit and learn from past attempts.
- **Research and analytics infrastructure** supporting longitudinal studies and exploratory insights.

This combination positions GATEWay as a strong candidate for:

- **Advanced undergraduate or graduate project submissions**: Demonstrating systems integration, machine learning, and educational technology depth.
- **Technical job portfolios**: Showcasing full-stack development, architectural thinking, and domain expertise.

- **Academic research**: Publishing work on adaptive learning, recommendation systems, or educational analytics.
- **StartUp/Product pitches**: Presenting a differentiated solution addressing real learner and educator needs.
- **Viva and technical interviews**: Discussing complex systems, tradeoff decisions, and future research directions.

The platform demonstrates that educational software, when thoughtfully designed, can elevate not just test scores but learner autonomy, teacher effectiveness, and the quality of educational evidence.